

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



Báo cáo kết quả huấn luyện mô hình

Đề tài: Hệ thống phát hiện vi phạm không đội mũ bảo hiểm

Môn học: Nhập môn học máy

Sinh viên thực hiện:

Đỗ Tiến Đạt - 23120119

Lê Nhật Minh Tâm - 23120163

Triệu Tuấn Kiệt - 23120137

Nguyễn Đăng Khoa - 23120134

Huỳnh Mạnh Tường - 23120105

Giáo viên hướng dẫn:

Thầy Bùi Tiến Lên

Ngày 21 tháng 7 năm 2026

Thông tin nhóm và Phân công công việc

Họ và tên	MSSV	Phân công công việc
Đỗ Tiến Đạt	23120119	Trưởng nhóm, phụ trách quản lý chung và thiết kế hệ thống.
Lê Nhật Minh Tâm	23120163	Phụ trách thu thập dữ liệu, tiền xử lý và phân tích khám phá dữ liệu (EDA).
Triệu Tuấn Kiệt	23120137	Phụ trách nghiên cứu kiến trúc, lựa chọn và huấn luyện mô hình.
Nguyễn Đăng Khoa	23120134	Phụ trách phát triển ứng dụng Web (giao diện, các chức năng và tích hợp API).
Huỳnh Mạnh Tường	23120105	Phụ trách đóng gói mô hình, thiết lập môi trường và triển khai ứng dụng.

Bảng 1: Bảng phân công công việc và đóng góp của các thành viên

Mục lục

Thông tin nhóm và Phân công công việc	i
1 Giới thiệu	1
1.1 Phân tích Vấn đề (Problem Definition)	1
1.2 Mục tiêu của Đề án	1
1.3 Tổng quan về Phương pháp	2
2 Thu thập và Phân tích Dữ liệu	3
2.1 Nguồn và Phương pháp Thu thập	3
2.2 Tiền xử lý và Làm sạch	3
2.3 Phân tích Khám phá Dữ liệu (EDA)	3
3 Lựa chọn và Huấn luyện Mô hình	4
3.1 Chuẩn bị Dữ liệu cho Mô hình	4
3.2 Lựa chọn và Kiến trúc Mô hình	5
3.2.1 YOLO	5
3.2.2 Faster R-CNN	6
3.2.3 RT-DETR	7
3.3 Cấu hình Huấn luyện	8
4 Kết quả và Thảo luận	10
4.1 Kết quả Thực nghiệm	10
4.1.1 YOLO	10
4.1.2 Faster R-CNN	11
4.1.3 RT-DETR	13
4.2 So sánh và Thảo luận	14
4.2.1 Phân tích overfitting và underfitting	16
4.2.2 Phân tích lỗi dự đoán	16
5 Xây dựng và Triển khai Ứng dụng	17
5.1 Kiến trúc Hệ thống	17

5.2	Giao diện và Chức năng	18
5.2.1	Dăng nhập và phân quyền người dùng	18
5.2.2	Bảng điều khiển tổng quan	19
5.2.3	Xem kết quả phát hiện và kiểm duyệt vi phạm	20
5.2.4	Phát hiện AI theo thời gian thực	21
5.3	Triển khai	22
6	Kết luận	23
6.1	Tóm tắt Kết quả	23
6.2	Hạn chế	23
6.3	Hướng phát triển	23
	Tài liệu tham khảo	24

Danh sách bảng

1	Bảng phân công công việc và đóng góp của các thành viên	i
2	Tỷ lệ chia dữ liệu dùng cho huấn luyện và đánh giá	4
3	So sánh ba mô hình đã thử nghiệm	5
4	Cấu hình huấn luyện chính của ba mô hình	9
5	Chỉ số theo từng lớp trên test set (COCOeval; P/R/F1 tại $\text{conf} = 0.25$, $\text{IoU} = 0.5$)	14
6	Chỉ số gộp trên test set (tham chiếu; FPS đo trên GPU Tesla T4)	14

Danh sách hình vẽ

1	Sơ đồ tổng quan quy trình xây dựng hệ thống phát hiện vi phạm mũ bảo hiểm	3
2	Sơ đồ kiến trúc tổng quát của YOLO trong bài toán phát hiện mũ bảo hiểm.	6
3	Sơ đồ kiến trúc Faster R-CNN với ResNet50-FPN.	7
4	Sơ đồ kiến trúc RT-DETR dùng backbone, hybrid encoder và transformer decoder. .	8
5	Learning curves và validation metrics của YOLO.	10
6	Confusion matrix chuẩn hóa và Precision-Recall curve của YOLO.	11
7	Learning curves và learning-rate schedule của Faster R-CNN được sinh từ số liệu huấn luyện.	12
8	Confusion matrix chuẩn hóa của Faster R-CNN trên test set.	12
9	Learning curves và validation metrics của RT-DETR trong lần huấn luyện resume. .	13
10	Confusion matrix chuẩn hóa và Precision-Recall curve của RT-DETR.	13
11	So sánh mAP, recall/AR và FPS giữa ba mô hình trên test set.	15
12	AP@0.5 tách theo từng lớp. Faster R-CNN vượt trội ở <i>motorbike</i> nhưng thua ở <i>non-helmet</i>	15
13	Kiến trúc triển khai của hệ thống phát hiện vi phạm không đội mũ bảo hiểm. . . .	17
14	Giao diện đăng nhập của hệ thống.	19
15	Bảng điều khiển tổng quan thống kê vi phạm và trạng thái phê duyệt.	20
16	Màn hình xem kết quả phát hiện, dòng thời gian vi phạm và phê duyệt ảnh minh chứng.	21
17	Chế độ phát hiện AI hiển thị bounding box và thông tin suy luận trên video. . . .	22

1 Giới thiệu

1.1 Phân tích Vấn đề (Problem Definition)

Vi phạm không đội mũ bảo hiểm khi tham gia giao thông bằng xe máy là một trong những nguyên nhân hàng đầu làm gia tăng mức độ nghiêm trọng của tai nạn giao thông tại Việt Nam, nơi xe máy chiếm tỷ trọng phương tiện lưu thông áp đảo. Việc giám sát và xử phạt vi phạm này hiện chủ yếu dựa vào lực lượng cảnh sát giao thông quan sát trực tiếp hoặc rà soát thủ công camera an ninh, dẫn đến hạn chế về phạm vi bao phủ, tốn nhiều nhân lực và dễ bỏ sót vi phạm, đặc biệt tại các nút giao thông đông đúc hoặc vào khung giờ cao điểm.

Sự phát triển của các mô hình phát hiện đối tượng (object detection) dựa trên học sâu, kết hợp với hạ tầng camera giám sát giao thông sẵn có tại các đô thị lớn, mở ra khả năng xây dựng một hệ thống tự động phát hiện vi phạm không đội mũ bảo hiểm theo thời gian thực. Một hệ thống như vậy không chỉ giảm tải công việc giám sát thủ công mà còn cung cấp dữ liệu định lượng, minh chứng trực quan (hình ảnh/video vi phạm) hỗ trợ công tác xử phạt, góp phần nâng cao ý thức chấp hành luật giao thông và giảm thiểu thương vong do tai nạn.

1.2 Mục tiêu của Đề án

Đề án hướng tới các mục tiêu cụ thể sau:

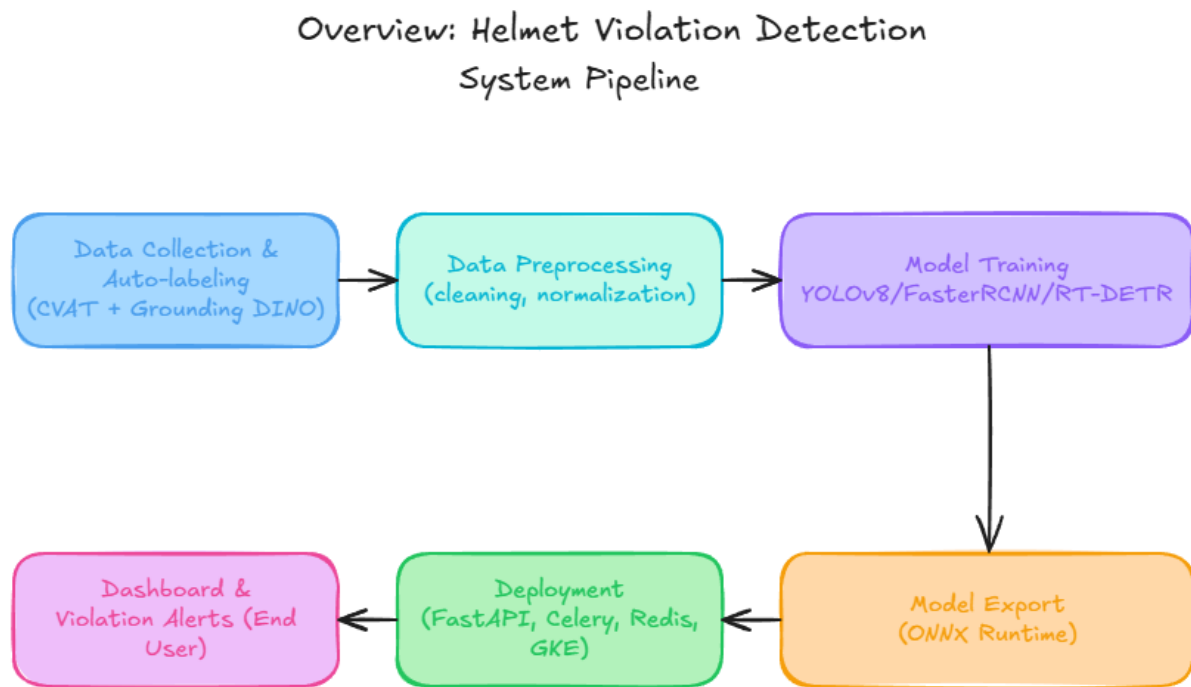
- Xây dựng bộ dữ liệu hình ảnh giao thông có gắn nhãn cho bài toán phát hiện người điều khiển xe máy không đội mũ bảo hiểm.
- Huấn luyện mô hình phát hiện đối tượng dựa trên kiến trúc **YOLOv8**, đạt độ chính xác và tốc độ suy luận phù hợp cho ứng dụng thời gian thực (real-time/near-real-time).
- Tối ưu hóa mô hình để triển khai hiệu quả thông qua **ONNX Runtime**, đảm bảo khả năng phục vụ suy luận với độ trễ thấp.
- Xây dựng và triển khai một hệ thống ứng dụng hoàn chỉnh theo kiến trúc microservices, cho phép tiếp nhận luồng dữ liệu (ảnh/video), thực hiện suy luận, lưu trữ kết quả và hiển thị vi phạm qua giao diện dashboard trực quan.

- Đánh giá hiệu năng mô hình trên tập dữ liệu kiểm thử, so sánh với các baseline, đồng thời phân tích hạn chế và đề xuất hướng cải thiện.

1.3 Tổng quan về Phương pháp

Phương pháp tiếp cận của đề án được triển khai theo một quy trình gồm các giai đoạn chính (xem Hình 1):

1. **Thu thập và gán nhãn dữ liệu:** Thu thập hình ảnh/video giao thông, thực hiện gán nhãn bán tự động bằng công cụ CVAT kết hợp mô hình Grounding DINO để tăng tốc quá trình tạo nhãn cho tập dữ liệu huấn luyện. Cuối cùng, nhóm kiểm tra thủ công toàn bộ nhãn bằng CVAT.
2. **Tiền xử lý dữ liệu:** Làm sạch, chuẩn hóa và tăng cường dữ liệu (data augmentation) nhằm cải thiện khả năng tổng quát hóa của mô hình.
3. **Huấn luyện mô hình:** Huấn luyện mô hình phát hiện đối tượng YOLOv8 trên tập dữ liệu đã xử lý, tinh chỉnh siêu tham số để tối ưu hiệu năng.
4. **Xuất và tối ưu mô hình:** Chuyển đổi mô hình sang định dạng ONNX, sử dụng ONNX Runtime để tối ưu tốc độ suy luận khi triển khai.
5. **Triển khai ứng dụng:** Đóng gói mô hình vào một hệ thống microservices (FastAPI, Celery, Redis, Next.js, Supabase) và triển khai trên nền tảng Google Kubernetes Engine (GKE), cho phép xử lý bất đồng bộ các tác vụ suy luận.
6. **Hiển thị và cảnh báo:** Kết quả phát hiện vi phạm được hiển thị qua giao diện dashboard, hỗ trợ quy trình xem xét và xác minh vi phạm.



Hình 1: Sơ đồ tổng quan quy trình xây dựng hệ thống phát hiện vi phạm mũ bảo hiểm

2 Thu thập và Phân tích Dữ liệu

2.1 Nguồn và Phương pháp Thu thập

Trình bày nguồn dữ liệu sử dụng cho bài toán phát hiện vi phạm không đội mũ bảo hiểm (tự thu thập, API, web scraping, các bộ dữ liệu công khai,...). Mô tả chi tiết quy trình thu thập, các công cụ sử dụng và định dạng lưu trữ của dữ liệu thô.

2.2 Tiền xử lý và Làm sạch

Mô tả chi tiết các bước xử lý nhiễu (noise), bổ sung dữ liệu thiếu (missing values), gán nhãn, chuẩn hóa kích thước ảnh/dữ liệu và các phép biến đổi dữ liệu cần thiết trước khi huấn luyện.

2.3 Phân tích Khám phá Dữ liệu (EDA)

Thực hiện thống kê mô tả bộ dữ liệu. Phân tích phân bố nhãn (ví dụ số lượng mẫu đội mũ bảo hiểm vs không đội mũ), mối quan hệ giữa các thuộc tính. Sử dụng các biểu đồ (bar chart, histogram, v.v.) và bảng biểu để minh họa các phát hiện quan trọng về dữ liệu. Phân tích các vấn đề về chất lượng

dữ liệu như dữ liệu ngoại lai (outliers), trùng lặp và cách giải quyết.

3 Lựa chọn và Huấn luyện Mô hình

3.1 Chuẩn bị Dữ liệu cho Mô hình

Sau bước thu thập, làm sạch và chuẩn hóa dữ liệu ở Chương 2, toàn bộ dữ liệu được tổ chức theo định dạng COCO và được chia thành ba tập độc lập: train, validation và test. Tập train dùng để cập nhật trọng số mô hình, tập validation dùng để theo dõi quá trình học và chọn siêu tham số, còn tập test chỉ được sử dụng ở cuối để đánh giá khả năng tổng quát hóa.

Bảng 2: Tỷ lệ chia dữ liệu dùng cho huấn luyện và đánh giá

Tập	Số ảnh	Tỷ lệ	motorbike	helmet	non-helmet	Vai trò
Train	2309	70.0%	3408	6739	900	Học đặc trưng và cập nhật trọng số mô hình.
Validation	330	10.0%	906	450	533	Chọn checkpoint, theo dõi overfitting và tinh chỉnh siêu tham số.
Test	659	20.0%	467	1256	138	Đánh giá cuối cùng trên dữ liệu không tham gia huấn luyện.

Tỷ lệ 70/10/20 được chọn để giữ phần lớn dữ liệu cho huấn luyện, đồng thời vẫn dành một tập test đủ lớn để đánh giá khách quan. Với bài toán phát hiện vi phạm không đội mũ bảo hiểm, dữ liệu có độ đa dạng cao về góc chụp, khoảng cách camera, ánh sáng và mật độ xe. Vì vậy, việc giữ riêng 20% dữ liệu cho test giúp kết quả cuối phản ánh tốt hơn khả năng sử dụng mô hình trong bối cảnh thực tế.

Các bước tiền xử lý cuối cùng trước khi đưa vào mô hình gồm:

- Chuẩn hóa ảnh về kích thước đầu vào 640x640 để thống nhất giữa các mô hình.
- Giữ annotation ở định dạng COCO cho Faster R-CNN; chuyển bounding box từ dạng xywh sang xyxy khi nạp vào PyTorch.
- Với YOLO và RT-DETR, chuyển COCO annotation sang định dạng YOLO gồm `class_id`, tâm bbox và kích thước bbox đã chuẩn hóa về khoảng $[0, 1]$.
- Chuyển class id từ COCO 1-indexed sang 0-indexed khi dùng pipeline Ultralytics.
- Dùng train set đã gộp thêm pseudo-labels trong file `instances_train_merged.json` để tăng số lượng mẫu huấn luyện.

Trong quá trình huấn luyện, YOLO và RT-DETR dùng các phép tăng cường dữ liệu mặc định của Ultralytics như thay đổi màu HSV, dịch chuyển, co giãn, lật ngang và mosaic [6]. Faster R-CNN được huấn luyện trực tiếp trên ảnh tensor và annotation gốc, không dùng augmentation mạnh để giữ pipeline baseline ổn định và dễ so sánh.

3.2 Lựa chọn và Kiến trúc Mô hình

Nhóm thử nghiệm ba hướng phát hiện đối tượng đại diện cho ba nhóm kiến trúc khác nhau: YOLO cho detector một giai đoạn, Faster R-CNN cho detector hai giai đoạn, và RT-DETR cho detector dựa trên transformer. Cách chọn này giúp so sánh được sự đánh đổi giữa tốc độ, độ chính xác và độ phức tạp mô hình.

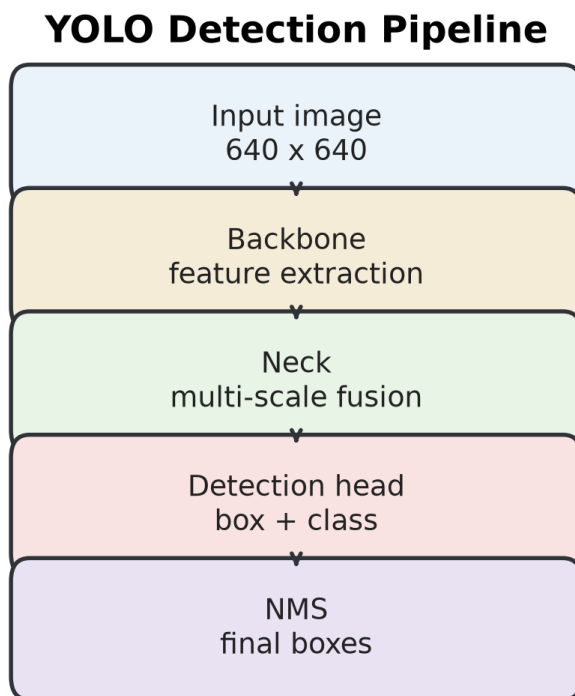
Bảng 3: So sánh ba mô hình đã thử nghiệm

Mô hình	Nhóm kiến trúc	Số tham số	Lý do thử nghiệm
YOLO	One-stage detector	21.78M	Tốc độ inference cao, phù hợp với hệ thống cần xử lý ảnh/video gần thời gian thực.
Faster R-CNN	Two-stage detector	41.31M	Baseline mạnh về độ chính xác nhờ tách bước sinh vùng đề xuất và bước phân loại/tính chỉnh bbox.
RT-DETR	Transformer detector	32.81M	Hướng tiếp cận mới hơn, dùng object queries và encoder-decoder để giảm phụ thuộc vào các bước hậu xử lý thủ công.

Dựa trên mục tiêu của hệ thống là phát hiện vi phạm từ ảnh hoặc video giao thông, YOLO được chọn làm ứng viên triển khai chính vì có tốc độ cao nhất trong các mô hình đã thử nghiệm, trong khi vẫn giữ mAP@0.5 và recall ở mức sử dụng được. Faster R-CNN được dùng như baseline độ chính xác để kiểm tra giới hạn chất lượng phát hiện, còn RT-DETR được dùng để đánh giá hướng transformer end-to-end.

3.2.1 YOLO

YOLO là detector một giai đoạn, dự đoán trực tiếp bounding box và lớp đối tượng từ feature map mà không cần tách riêng bước proposal [4]. Pipeline YOLO gồm backbone để trích xuất đặc trưng, neck để hợp nhất đặc trưng đa tỉ lệ, và detection head để dự đoán bbox, class probability và phân phối bbox. Với bài toán hiện tại, kiến trúc này phù hợp vì xe máy và vùng đầu xuất hiện ở nhiều kích thước khác nhau trong ảnh giao thông.



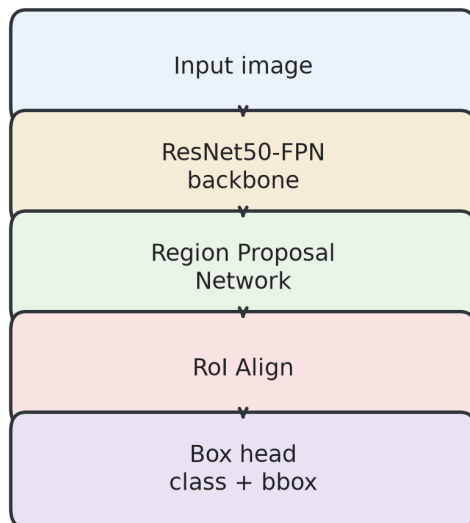
One-stage detector optimized for fast dense prediction.

Hình 2: Sơ đồ kiến trúc tổng quát của YOLO trong bài toán phát hiện mũ bảo hiểm.

3.2.2 Faster R-CNN

Faster R-CNN là detector hai giai đoạn [5]. Giai đoạn đầu dùng Region Proposal Network (RPN) để sinh các vùng có khả năng chứa đối tượng. Giai đoạn thứ hai dùng RoI Align/RoI head để phân loại proposal và tính chỉnh tọa độ bbox. Mô hình sử dụng backbone ResNet50-FPN pretrained [2]; FPN giúp mô hình khai thác đặc trưng ở nhiều tỉ lệ, hữu ích khi cần phát hiện cả xe máy lớn và vùng đầu nhỏ.

Faster R-CNN Detection Pipeline



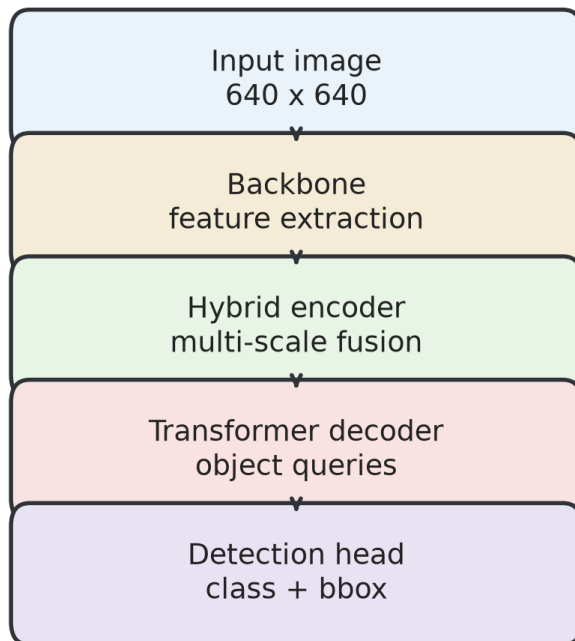
Two-stage detector: proposals followed by classification and box regression.

Hình 3: Sơ đồ kiến trúc Faster R-CNN với ResNet50-FPN.

3.2.3 RT-DETR

RT-DETR là detector dựa trên transformer được thiết kế cho phát hiện đối tượng thời gian thực [7]. Mô hình gồm backbone, hybrid encoder để hợp nhất đặc trưng đa tỉ lệ, transformer decoder để xử lý object queries và detection head để dự đoán class cùng bbox. Khác với YOLO, RT-DETR hướng tới dự đoán end-to-end bằng tập object queries, nhờ đó giảm phụ thuộc vào thiết kế anchor hoặc hậu xử lý phức tạp.

RT-DETR Detection Pipeline



End-to-end transformer-based detector.

Hình 4: Sơ đồ kiến trúc RT-DETR dùng backbone, hybrid encoder và transformer decoder.

3.3 Cấu hình Huấn luyện

Cả ba mô hình đều được huấn luyện từ trọng số pretrained để tận dụng đặc trưng thị giác đã học từ tập dữ liệu lớn. Việc fine-tune từ pretrained giúp giảm thời gian huấn luyện và phù hợp với kích thước dữ liệu của đề án.

Bảng 4: Cấu hình huấn luyện chính của ba mô hình

Thành phần	YOLO	Faster R-CNN	RT-DETR
Checkpoint khởi tạo	yo1o26m.pt pretrained	ResNet50-FPN pretrained	pre-RT-DETR pretrained
Epochs	50	50	40
Batch size	32	4	2
Kích thước ảnh	640x640	640x640	640x640
Optimizer	Auto optimizer của Ultralytics	SGD, momentum 0.9	AdamW
Learning rate	0.000111	0.001799	0.000241
Weight decay	0.000124	0.0005	0.000139
Loss chính	Box loss, classification loss, DFL loss	RPN objectness/bbox loss và ROI classification/bbox loss	GIoU loss, classification loss, L1 box loss

Với YOLO, hàm mất mát gồm ba thành phần chính: box loss để tối ưu tọa độ bbox, classification loss để tối ưu nhãn lớp và Distribution Focal Loss (DFL) để cải thiện độ chính xác định vị bbox. Với Faster R-CNN, loss được tách theo hai giai đoạn: RPN loss cho objectness và regression của proposal, sau đó là ROI loss cho phân loại và tinh chỉnh bbox. Với RT-DETR, mô hình dùng tổ hợp GIoU loss, L1 box loss và classification loss để tối ưu đồng thời vị trí và lớp của object queries. Quá trình tinh chỉnh siêu tham số được thực hiện theo hai pha. Pha đầu sử dụng random search thông qua Ray Tune để thử nhiều tổ hợp learning rate, batch size, weight decay và hệ số suy giảm learning rate. Pha thứ hai dùng cấu hình tốt nhất trên validation set để train final model và lưu checkpoint tốt nhất.

Cấu hình tốt nhất sau bước tinh chỉnh được tóm tắt như sau:

- YOLO: learning rate 0.000111, batch size 32, LRF 0.177187 và weight decay 0.000124.
- RT-DETR: learning rate 0.000241, batch size 2, LRF 0.032969, weight decay 0.000139 và optimizer AdamW.
- Faster R-CNN: tinh chỉnh learning rate, batch size và weight decay quanh cấu hình SGD ổn định, sau đó chọn checkpoint dựa trên validation loss.

Các chỉ số validation được dùng để theo dõi quá trình học, còn chỉ số test chỉ được dùng sau cùng để so sánh mô hình. Cách tổ chức này giúp giảm nguy cơ chọn mô hình theo test set và đảm bảo kết quả ở Chương 4 phản ánh tương đối khách quan hiệu năng của từng kiến trúc.

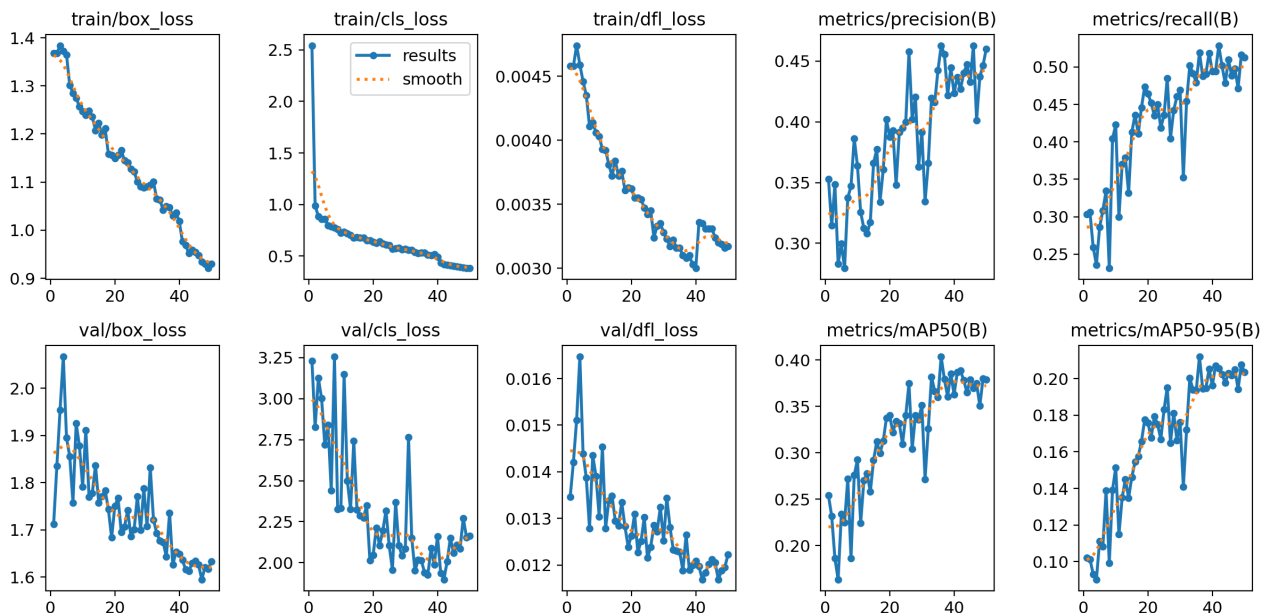
4 Kết quả và Thảo luận

4.1 Kết quả Thực nghiệm

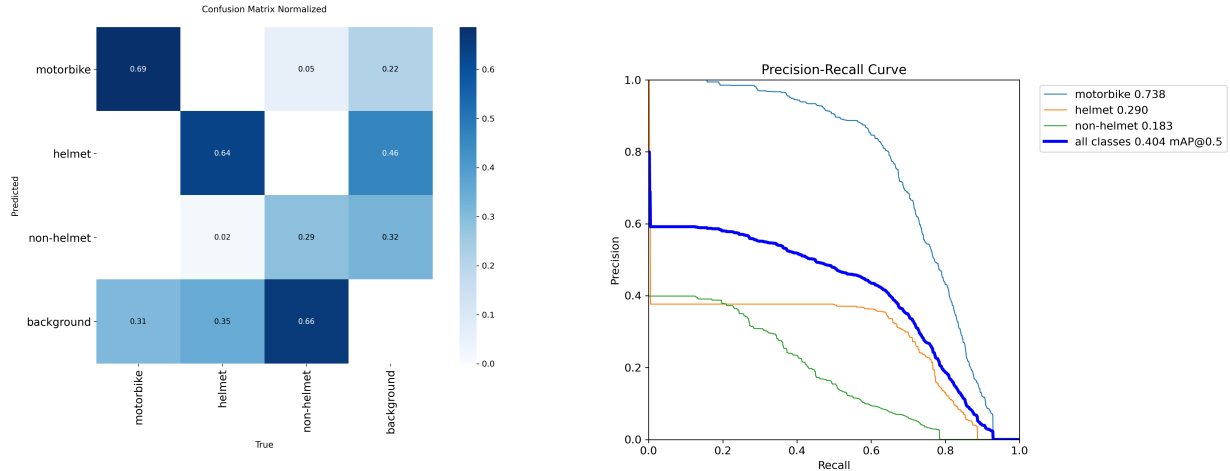
Sau khi huấn luyện, nhóm đánh giá ba mô hình trên cùng tập validation và test độc lập. Với bài toán object detection, các chỉ số chính được sử dụng gồm $mAP@0.5$, $mAP@0.5:0.95$, recall/AR và tốc độ inference. $mAP@0.5$ phản ánh khả năng phát hiện đúng ở ngưỡng IoU tương đối phổ biến, trong khi $mAP@0.5:0.95$ nghiêm ngặt hơn vì trung bình trên nhiều ngưỡng IoU. Do hệ thống hướng tới phát hiện vi phạm trong ảnh/video giao thông, recall/AR và FPS cũng được xem là các chỉ số quan trọng.

4.1.1 YOLO

YOLO được huấn luyện trong 50 epoch. Hình 5 cho thấy train loss giảm ổn định, đặc biệt classification loss giảm mạnh ở giai đoạn đầu. Trên validation, $mAP@0.5:0.95$ đạt tốt nhất tại epoch 36 với giá trị 0.2121; cùng epoch này $mAP@0.5$ đạt 0.4037, precision đạt 0.4628 và recall đạt 0.5192. Sau epoch 36, metric validation dao động nhẹ, cho thấy mô hình đã gần đạt trạng thái hội tụ.



Hình 5: Learning curves và validation metrics của YOLO.

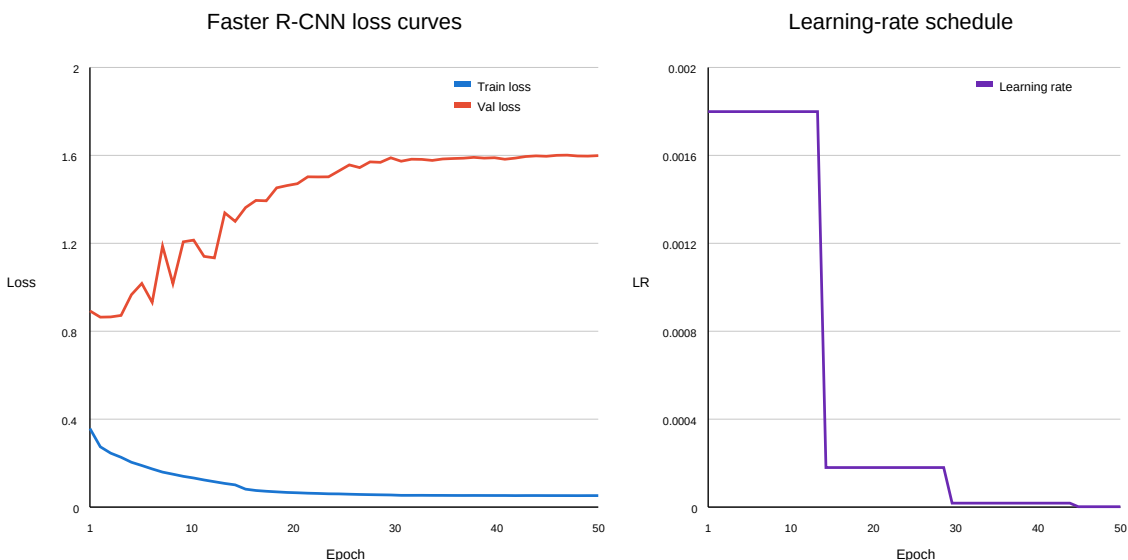


Hình 6: Confusion matrix chuẩn hóa và Precision-Recall curve của YOLO.

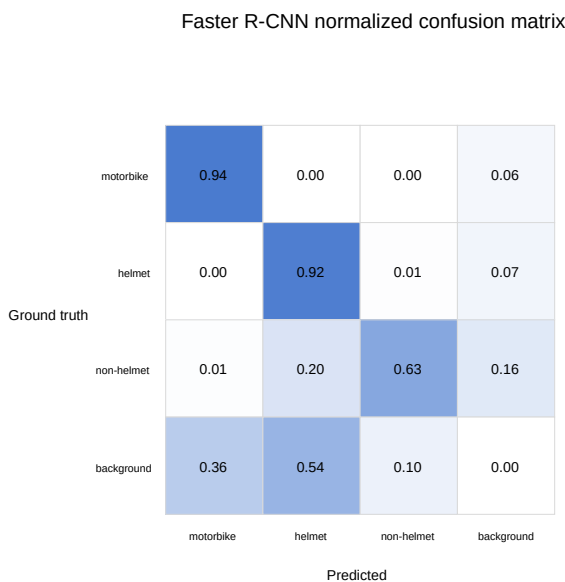
Trên test set, YOLO đạt $mAP@0.5$ là 0.582 và $mAP@0.5:0.95$ là 0.389. Quan trọng hơn, ở lớp *non-helmet* mô hình đạt $AP@0.5$ là 0.636 với F1 là 0.611 — cao nhất trong ba mô hình — nhờ precision 0.613 cân bằng với recall 0.609. Đây cũng là mô hình có tốc độ inference cao nhất, đạt 36.4 FPS, phù hợp với mục tiêu triển khai hệ thống xử lý ảnh/video theo thời gian thực.

4.1.2 Faster R-CNN

Faster R-CNN được huấn luyện trong 50 epoch với backbone ResNet50-FPN. Vì thư mục kết quả Faster R-CNN không có sẵn biểu đồ, nhóm sinh lại các hình trong Hình 7 từ file learning curve CSV và JSON kết quả. Train loss giảm từ 0.3576 ở epoch 1 xuống 0.0523 ở epoch 50, cho thấy mô hình học rất mạnh trên train set. Tuy nhiên validation loss thấp nhất ở epoch 2 với giá trị 0.8642 rồi tăng dần đến 1.5990 ở epoch 50. Đây là dấu hiệu overfitting rõ rệt.



Hình 7: Learning curves và learning-rate schedule của Faster R-CNN được sinh từ số liệu huấn luyện.



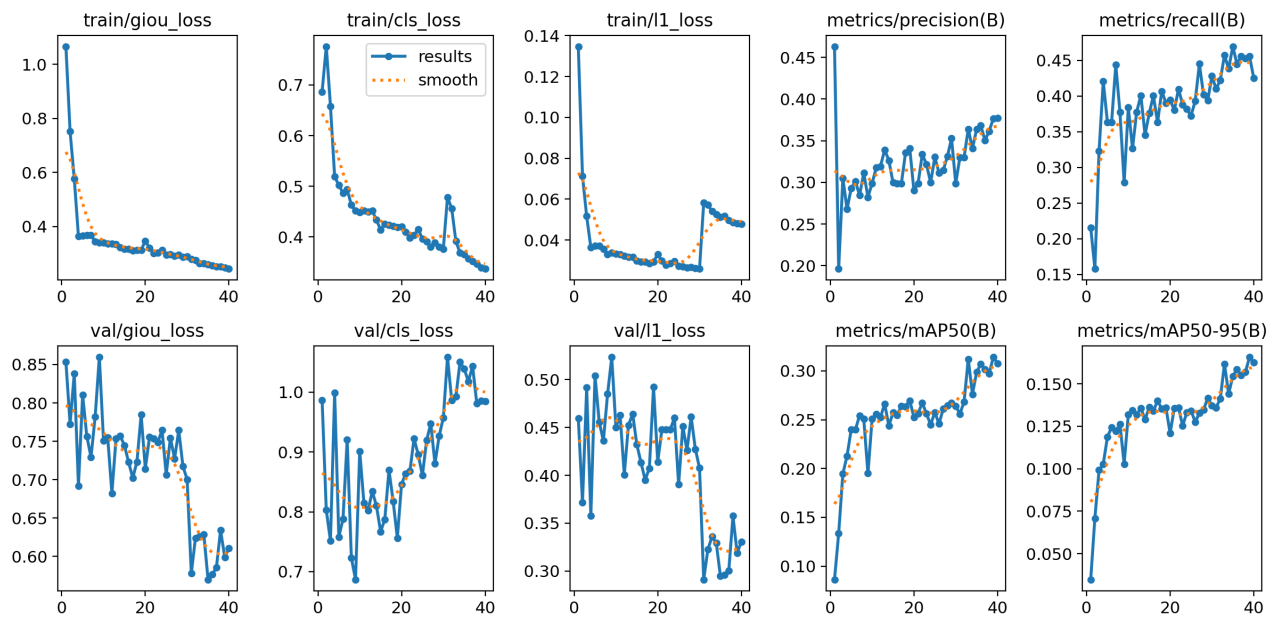
Hình 8: Confusion matrix chuẩn hóa của Faster R-CNN trên test set.

Trên test set, Faster R-CNN đạt $mAP@0.5$ là 0.650 và $mAP@0.5:0.95$ là 0.423 — cao nhất trong ba mô hình nếu chỉ nhìn con số gộp. Tuy nhiên ưu thế này đến gần như trọn vẹn từ lớp *motorbike* với $AP@0.5$ là 0.779. Ở lớp *non-helmet*, mô hình chỉ đạt $AP@0.5$ là 0.613 và F1 là 0.346, do precision

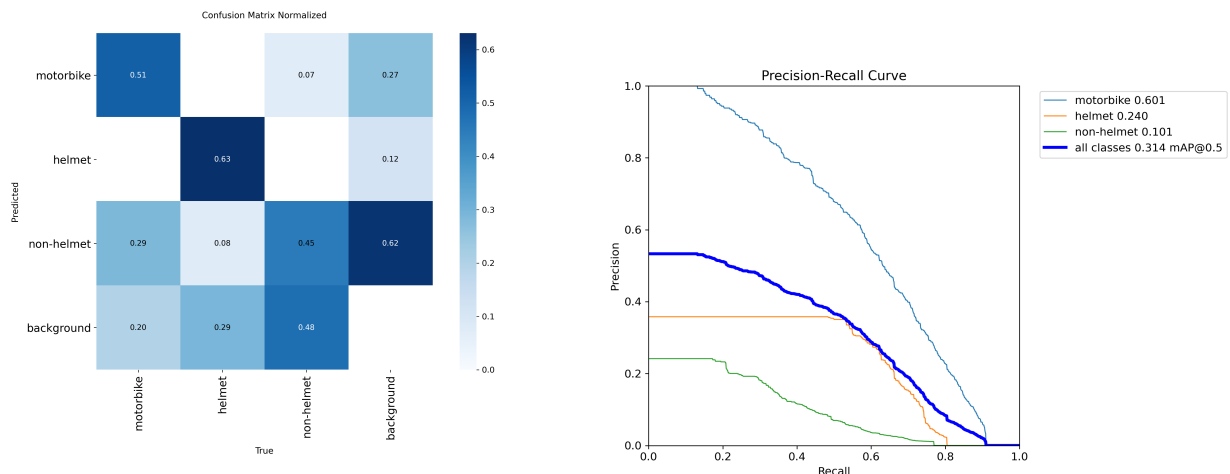
thấp (0.238) khiến sinh nhiều cảnh báo sai. Tốc độ đạt 12.4 FPS. Faster R-CNN vì vậy phù hợp làm baseline hai giai đoạn hơn là lựa chọn triển khai.

4.1.3 RT-DETR

RT-DETR được huấn luyện tiếp trong thư mục `rt detr_resume`. Hình 9 cho thấy train GIoU loss và L1 loss giảm đều, chứng tỏ phần định vị bbox được cải thiện qua các epoch. Validation mAP@0.5:0.95 đạt tốt nhất ở epoch 39 với giá trị 0.1657; tại epoch này mAP@0.5 đạt 0.3141, precision đạt 0.3769 và recall đạt 0.4558.



Hình 9: Learning curves và validation metrics của RT-DETR trong lần huấn luyện resume.



Hình 10: Confusion matrix chuẩn hóa và Precision-Recall curve của RT-DETR.

Trên test set, checkpoint RT-DETR final đạt $mAP@0.5$ là 0.555 và $mAP@0.5:0.95$ là 0.369. Điểm đáng chú ý là ở lớp *non-helmet*, RT-DETR đạt $AP@0.5$ cao nhất trong ba mô hình với 0.646 và recall cao nhất với 0.739, cho thấy khả năng tìm được nhiều vi phạm thật. Tuy nhiên precision chỉ 0.180 nên F1 rớt xuống 0.289, tức mô hình sinh rất nhiều cảnh báo sai ở ngưỡng hiện tại. Cộng thêm tốc độ chỉ 2.8 FPS, RT-DETR chưa phù hợp để triển khai thời gian thực nếu không hiệu chỉnh lại ngưỡng và tối ưu tốc độ.

4.2 So sánh và Thảo luận

Toàn bộ số liệu dưới đây được đo lại bằng script `models/checkpoints/eval_per_class.py` trên cùng tập test gồm 659 ảnh và 1861 annotation, dùng chung một bộ đánh giá COCOeval cho cả ba mô hình. Precision, Recall và F1 tính tại ngưỡng vận hành $confidence = 0.25$ và $IoU = 0.5$.

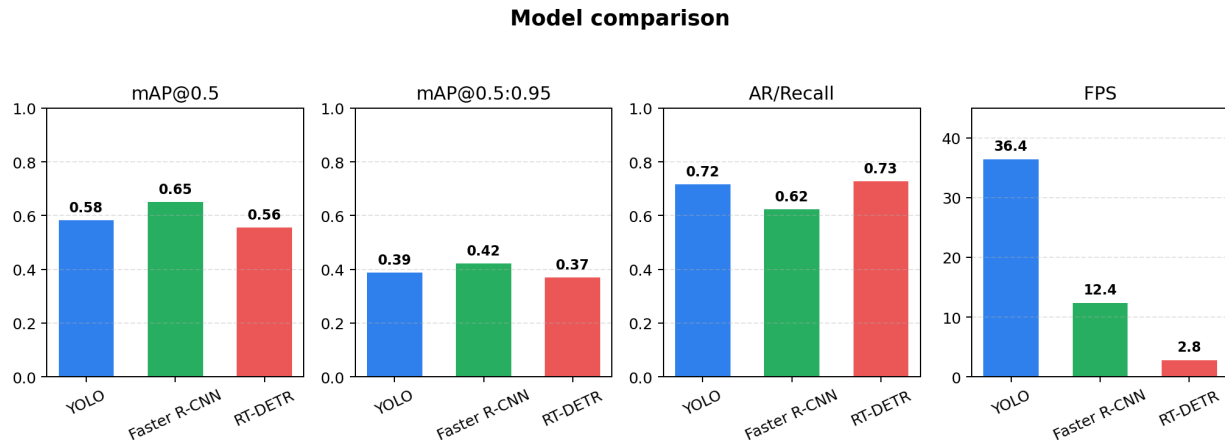
Tập test mất cân bằng nặng: *helmet* có 1256 bounding box, *motorbike* có 467, còn *non-helmet* — lớp quyết định của bài toán — chỉ có 138 box, khoảng 7% tổng số annotation. Vì mAP là trung bình cộng theo lớp với trọng số bằng nhau, một mô hình mạnh ở hai lớp dễ vẫn có thể đạt mAP cao dù yếu ở đúng lớp cần quan tâm. Do đó báo cáo trình bày chỉ số **tách riêng theo từng lớp**.

Bảng 5: Chỉ số theo từng lớp trên test set (COCOeval; P/R/F1 tại $conf = 0.25$, $IoU = 0.5$)

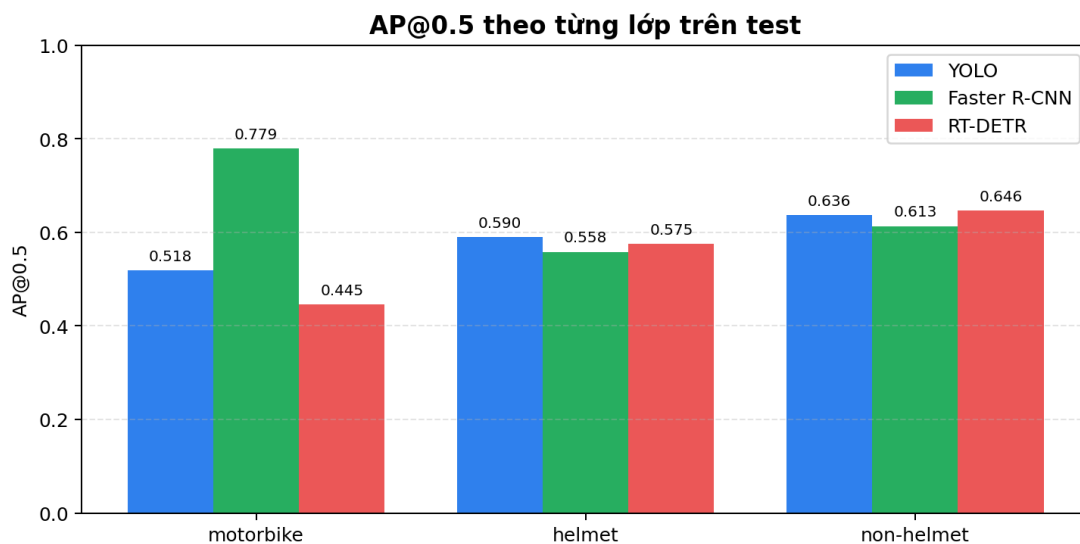
Mô hình	Lớp	$AP@0.5$	$AP@0.5:0.95$	P	R	F1
YOLO	motorbike	0.518	0.422	0.385	0.947	0.547
	helmet	0.590	0.329	0.588	0.935	0.722
	non-helmet	0.636	0.417	0.613	0.609	0.611
Faster R-CNN	motorbike	0.779	0.602	0.311	0.944	0.468
	helmet	0.558	0.271	0.442	0.920	0.597
	non-helmet	0.613	0.395	0.238	0.630	0.346
RT-DETR	motorbike	0.445	0.362	0.365	0.940	0.526
	helmet	0.575	0.315	0.533	0.928	0.677
	non-helmet	0.646	0.430	0.180	0.739	0.289

Bảng 6: Chỉ số gộp trên test set (tham chiếu; FPS đo trên GPU Tesla T4)

Mô hình	$mAP@0.5$	$mAP@0.5:0.95$	$AR@100$	FPS
YOLO	0.582	0.389	0.717	36.4
Faster R-CNN	0.650	0.423	0.624	12.4
RT-DETR	0.555	0.369	0.727	2.8



Hình 11: So sánh mAP, recall/AR và FPS giữa ba mô hình trên test set.



Hình 12: AP@0.5 tách theo từng lớp. Faster R-CNN vượt trội ở *motorbike* nhưng thua ở *non-helmet*.

Nếu chỉ nhìn Bảng 6, kết luận sẽ là Faster R-CNN tốt nhất vì đạt mAP@0.5 cao nhất (0.650) và mAP@0.5:0.95 cao nhất (0.423). Tuy nhiên Bảng 5 cho thấy con số gộp này gây hiểu nhầm: ưu thế của Faster R-CNN gần như đến trọn vẹn từ lớp *motorbike*, nơi nó đạt $AP@0.5 = 0.779$ trong khi hai mô hình còn lại chỉ quanh 0.45–0.52. Đây là lớp dễ nhất — vật thể lớn, hình dạng rõ, nhiều mẫu — và cũng là lớp ít giá trị nhất với mục tiêu phát hiện vi phạm. Ngược lại, ở lớp *non-helmet*, Faster R-CNN chỉ đạt $AP@0.5 = 0.613$, thấp hơn cả YOLO (0.636) lẫn RT-DETR (0.646).

Khác biệt rõ rệt nhất nằm ở điểm vận hành thực tế. Tại ngưỡng confidence 0.25, F1 của lớp *non-helmet* là 0.611 với YOLO, nhưng chỉ 0.346 với Faster R-CNN và 0.289 với RT-DETR. Nguyên nhân là precision: YOLO đạt 0.613, trong khi Faster R-CNN chỉ 0.238 và RT-DETR chỉ 0.180. Nói cách khác, RT-DETR bắt được nhiều vi phạm nhất (recall 0.739) nhưng cứ khoảng năm cảnh báo

thì bốn cái là báo nhầm, khiến chi phí kiểm duyệt thủ công rất lớn. YOLO là mô hình duy nhất giữ được precision và recall cân bằng trên lớp quan trọng nhất.

4.2.1 Phân tích overfitting và underfitting

YOLO có quá trình học ổn định nhất: train loss giảm đều và validation metric tăng dần trước khi dao động nhẹ ở cuối. Faster R-CNN có overfitting rõ nhất vì train loss tiếp tục giảm trong khi validation loss tăng sau epoch 2. RT-DETR nằm giữa hai trường hợp này: localization loss cải thiện, nhưng classification loss và validation metric vẫn dao động, cho thấy mô hình chưa thật sự ổn định trên các lớp nhỏ như *helmet* và *non-helmet*.

Nguyên nhân chính có thể đến từ ba yếu tố. Thứ nhất, dữ liệu mất cân bằng lớp; *helmet* xuất hiện nhiều hơn *non-helmet*. Thứ hai, vùng đầu thường nhỏ, mờ hoặc bị che khuất, khiến mô hình khó phân biệt có mũ và không mũ. Thứ ba, một phần ảnh có góc nhìn xa hoặc mật độ xe cao, làm tăng khả năng bỏ sót và sinh bbox thừa.

4.2.2 Phân tích lỗi dự đoán

Các lỗi dự đoán thường gặp gồm:

- **False negative:** mô hình bỏ sót vùng đầu nhỏ, người ở xa camera hoặc đối tượng bị che bởi xe/người khác.
- **False positive:** mô hình sinh bbox trên nền hoặc trên các vùng giống mũ/đầu người. Faster R-CNN có nhiều false positive hơn, thể hiện qua precision thấp hơn recall.
- **Nhầm lớp:** *helmet* và *non-helmet* dễ bị nhầm khi vùng đầu nhỏ, ảnh mờ, hoặc góc chụp từ sau/lệch bên.
- **Sai localization:** bbox đôi khi bao chưa sát vùng đầu hoặc xe máy, đặc biệt khi đối tượng bị cắt ở rìa ảnh.

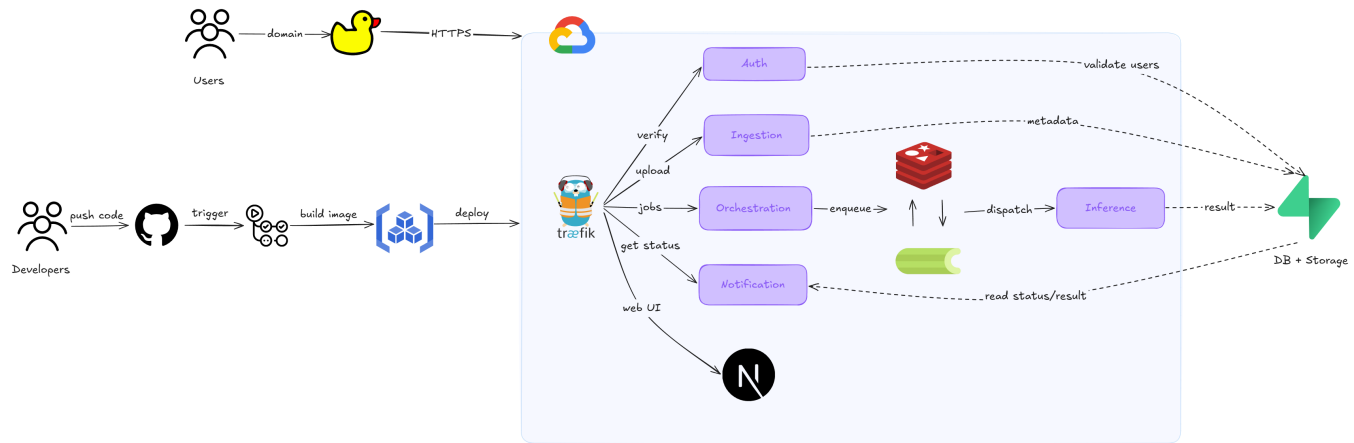
Tổng hợp lại, YOLO là mô hình phù hợp nhất cho triển khai, nhưng lý do không phải vì nó dẫn đầu mọi chỉ số gộp — thực tế Faster R-CNN mới có mAP cao nhất. Lý do là YOLO cho chất lượng tốt nhất trên đúng lớp mà hệ thống hướng tới: F1 của *non-helmet* đạt 0.611, gấp gần hai lần Faster R-CNN (0.346) và RT-DETR (0.289), đồng thời chạy nhanh nhất với 36.4 FPS. RT-DETR

là hướng transformer có tiềm năng vì đạt AP và recall cao nhất cho *non-helmet*, nhưng precision chỉ 0.180 và tốc độ 2.8 FPS nên cần hiệu chỉnh ngưỡng và tối ưu tốc độ. Faster R-CNN vẫn là baseline hai giai đoạn hữu ích, song ưu thế của nó tập trung ở lớp *motorbike* thay vì lớp vi phạm.

5 Xây dựng và Triển khai Ứng dụng

5.1 Kiến trúc Hệ thống

Hệ thống phát hiện vi phạm không đội mũ bảo hiểm được xây dựng theo hướng ứng dụng web container hóa, gồm giao diện người dùng, các dịch vụ backend độc lập, hàng đợi xử lý và dịch vụ lưu trữ bên ngoài. Sơ đồ kiến trúc tổng thể được trình bày trong Hình 13.



Hình 13: Kiến trúc triển khai của hệ thống phát hiện vi phạm không đội mũ bảo hiểm.

Người dùng truy cập hệ thống thông qua tên miền DuckDNS. Tên miền này trỏ đến điểm vào công khai của hệ thống trên Google Cloud, sau đó các yêu cầu HTTPS được định tuyến bởi Traefik. Traefik đóng vai trò như cổng vào chung, tiếp nhận các yêu cầu từ bên ngoài và chuyển tiếp đến đúng thành phần bên trong cụm triển khai.

Phần giao diện được xây dựng bằng Next.js và được triển khai như một container riêng. Các yêu cầu hiển thị trang web được Traefik định tuyến đến frontend. Các yêu cầu nghiệp vụ như đăng nhập, tải ảnh hoặc video, tạo tác vụ phát hiện và theo dõi trạng thái xử lý được chuyển đến các dịch vụ backend tương ứng.

Backend được tách thành nhiều dịch vụ theo chức năng. Dịch vụ Auth phụ trách xác thực người dùng và làm việc với Supabase Auth. Dịch vụ Ingestion tiếp nhận dữ liệu đầu vào, lưu tệp tải lên và metadata vào Supabase. Dịch vụ Orchestration tạo và quản lý các tác vụ phát hiện, sau đó đưa

công việc vào Redis. Redis được sử dụng làm hàng đợi và bộ nhớ đệm trung gian. Inference Worker nhận tác vụ từ hàng đợi, chạy mô hình phát hiện vi phạm, sau đó ghi kết quả và đầu ra đã xử lý về Supabase. Dịch vụ Notification đọc trạng thái hoặc kết quả xử lý từ Supabase và cập nhật lại cho giao diện người dùng.

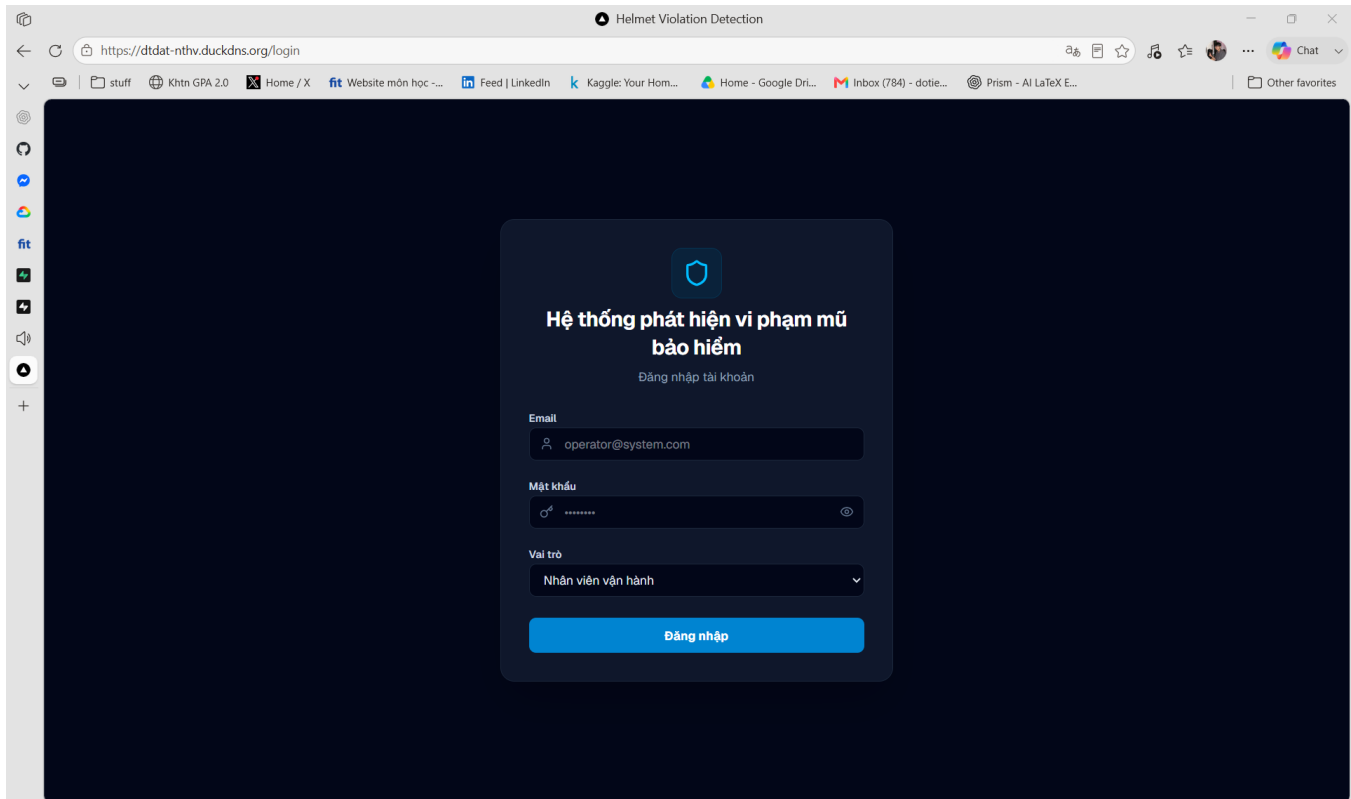
Supabase được đặt bên ngoài cụm Google Cloud và được sử dụng cho nhiều vai trò: cơ sở dữ liệu, xác thực, lưu trữ tệp và cập nhật thời gian thực. Cách tổ chức này giúp phần tính toán và phần triển khai ứng dụng nằm trên Google Cloud, trong khi dữ liệu người dùng, tệp đầu vào và kết quả được quản lý tập trung bởi Supabase.

5.2 Giao diện và Chức năng

Sau khi triển khai, hệ thống cung cấp giao diện web cho người vận hành theo dõi, tải lên và kiểm duyệt các kết quả phát hiện vi phạm. Các màn hình chính được thiết kế theo tông màu tối nhằm tăng khả năng quan sát video, ảnh minh chứng và các trạng thái cảnh báo trong thời gian dài.

5.2.1 Đăng nhập và phân quyền người dùng

Màn hình đăng nhập là điểm vào đầu tiên của hệ thống, cho phép người dùng nhập email, mật khẩu và chọn vai trò trước khi truy cập các chức năng nghiệp vụ. Trong phạm vi triển khai hiện tại, hệ thống hỗ trợ vai trò nhân viên vận hành để theo dõi dashboard, tải video, xem kết quả phát hiện và phê duyệt các trường hợp vi phạm. Việc xác thực người dùng được tích hợp với Supabase Auth, nhờ đó frontend chỉ hiển thị các trang nghiệp vụ sau khi người dùng có phiên đăng nhập hợp lệ.

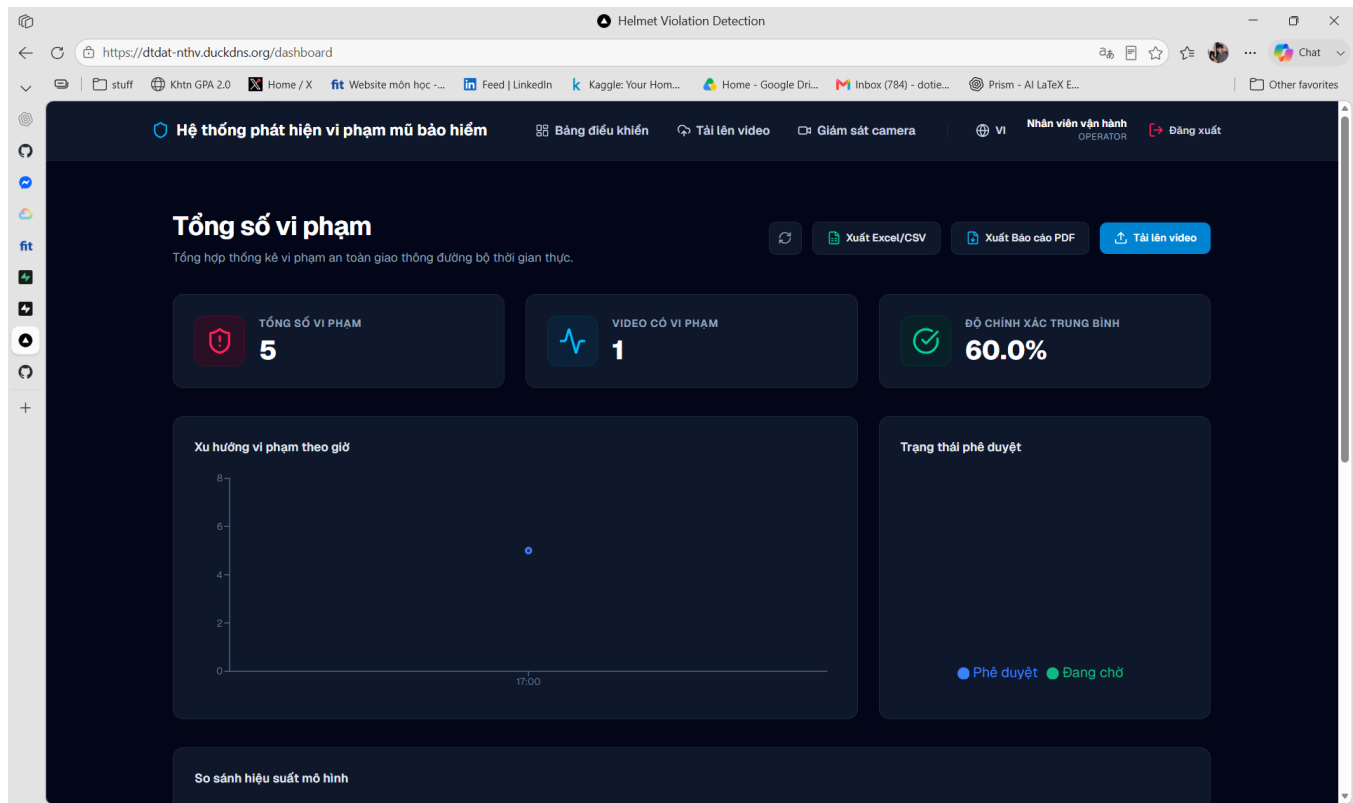


Hình 14: Giao diện đăng nhập của hệ thống.

5.2.2 Bảng điều khiển tổng quan

Sau khi đăng nhập, người dùng được chuyển đến bảng điều khiển tổng quan như Hình 15. Trang này tổng hợp các chỉ số chính của hệ thống, bao gồm tổng số vi phạm được phát hiện, số video có vi phạm và độ chính xác trung bình dựa trên các kết quả đã được người vận hành phê duyệt. Ngoài các thẻ thống kê, dashboard còn hiển thị xu hướng vi phạm theo thời gian và trạng thái phê duyệt, giúp người vận hành nhanh chóng nắm được tình hình xử lý dữ liệu.

Giao diện dashboard cũng cung cấp các thao tác nghiệp vụ thường dùng như làm mới dữ liệu, xuất báo cáo Excel/CSV, xuất báo cáo PDF và chuyển nhanh đến màn hình tải video. Thanh điều hướng phía trên cho phép di chuyển giữa các chức năng chính: bảng điều khiển, tải lên video, giám sát camera, đổi ngôn ngữ và đăng xuất.

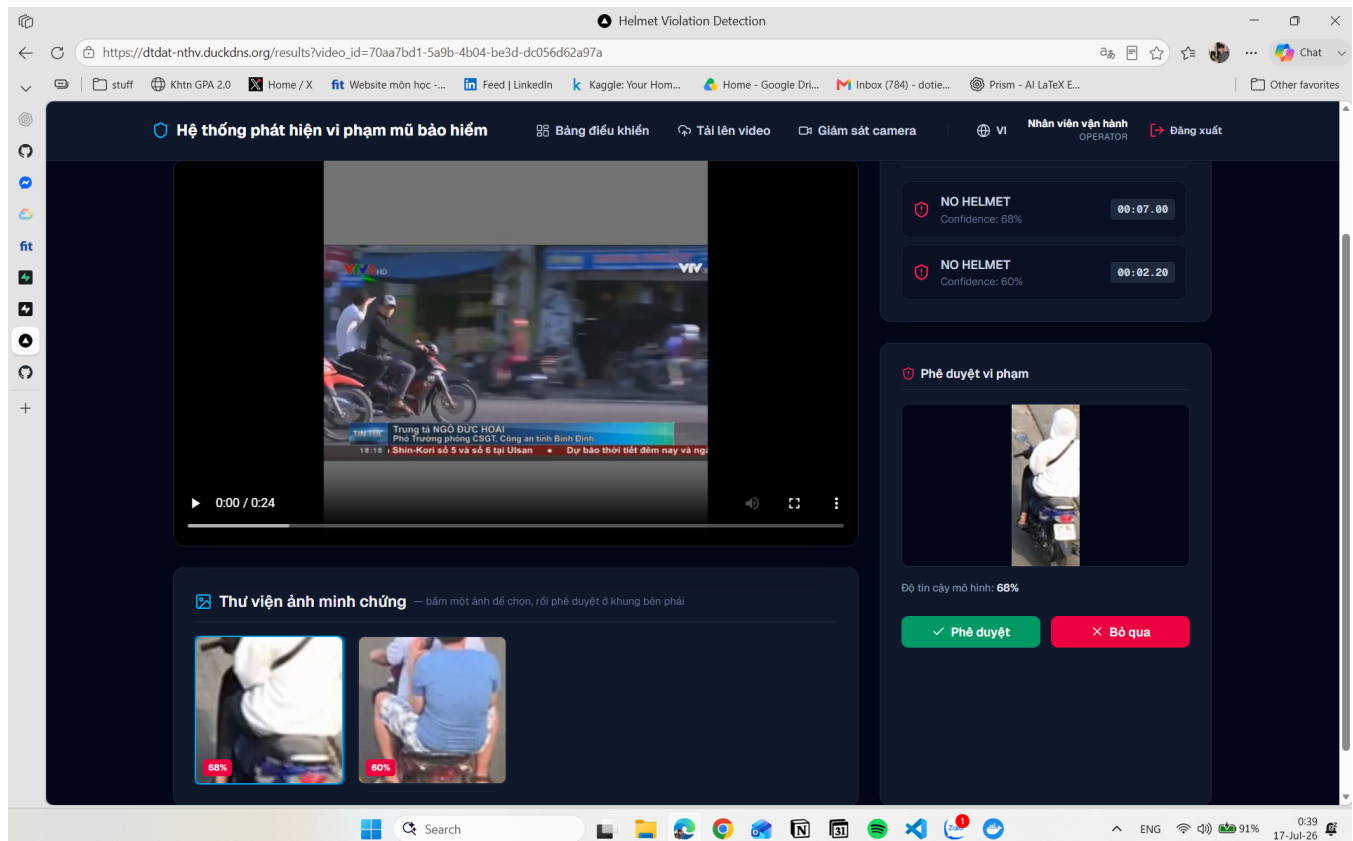


Hình 15: Bảng điều khiển tổng quan thống kê vi phạm và trạng thái phê duyệt.

5.2.3 Xem kết quả phát hiện và kiểm duyệt vi phạm

Màn hình kết quả phát hiện cho phép người vận hành xem lại video gốc, danh sách thời điểm có vi phạm và thư viện ảnh minh chứng được trích xuất từ quá trình suy luận. Ở khu vực bên phải, hệ thống hiển thị dòng thời gian vi phạm, trong đó mỗi mục gồm nhãn vi phạm, độ tin cậy của mô hình và mốc thời gian tương ứng trong video. Khi chọn một mục trong dòng thời gian, video được đưa đến đúng thời điểm đó để người vận hành kiểm tra trực quan.

Bên dưới khung video là thư viện ảnh minh chứng. Mỗi ảnh tương ứng với một phát hiện của mô hình và được gắn kèm độ tin cậy. Khi chọn một ảnh, khung phê duyệt ở bên phải hiển thị ảnh lớn hơn cùng độ tin cậy của mô hình, sau đó người vận hành có thể chọn *Phê duyệt* hoặc *Bỏ qua*. Cơ chế này giúp tách rõ hai bước: mô hình tự động đề xuất vi phạm, còn quyết định cuối cùng được ghi nhận thông qua thao tác kiểm duyệt của con người. Chỉ số độ chính xác trung bình trên dashboard được tính từ số phát hiện được phê duyệt trên tổng số phát hiện vi phạm.

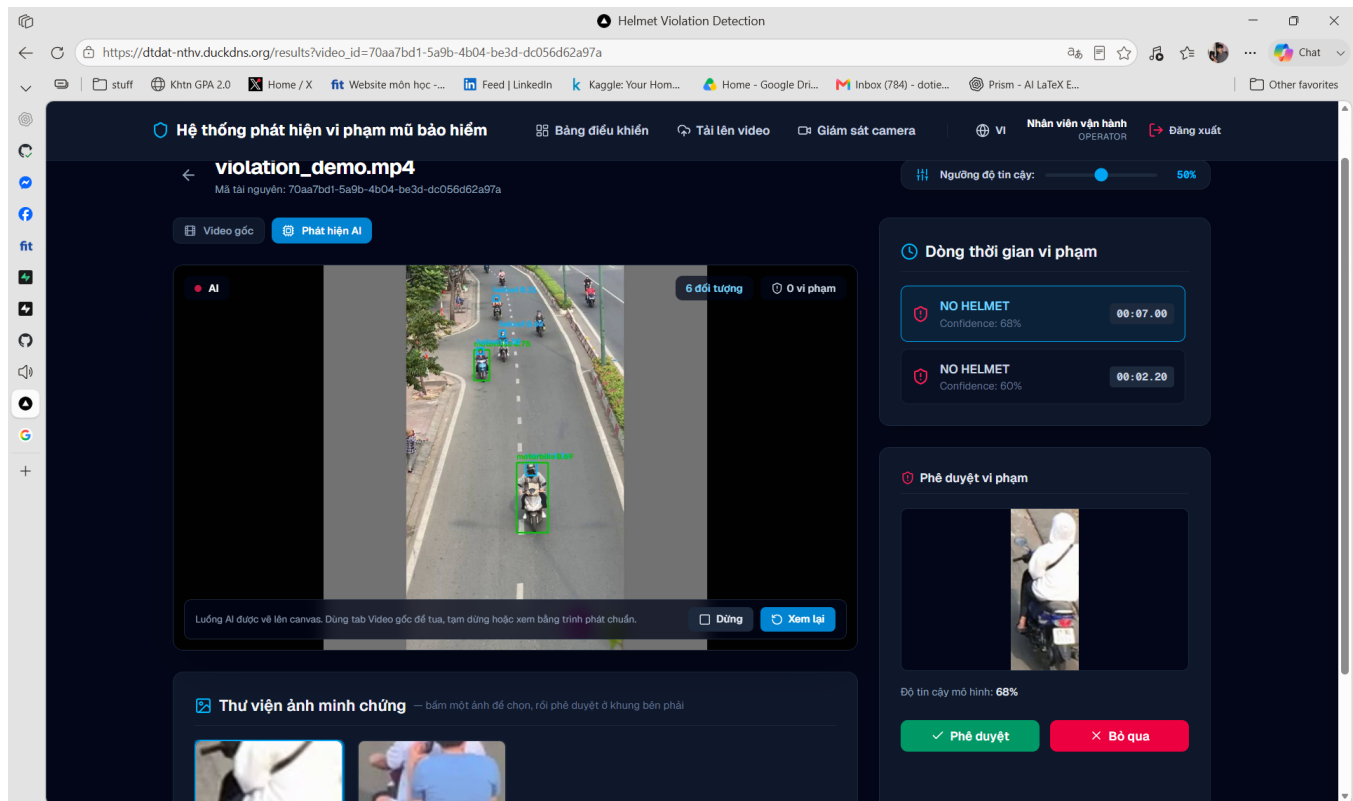


Hình 16: Màn hình xem kết quả phát hiện, đồng thời gian vi phạm và phê duyệt ảnh minh chứng.

5.2.4 Phát hiện AI theo thời gian thực

Ngoài chế độ xem video gốc, hệ thống cung cấp chế độ phát hiện AI để hiển thị luồng suy luận trực tiếp trên canvas. Ở chế độ này, các đối tượng được mô hình phát hiện được vẽ bằng bounding box kèm nhãn và độ tin cậy, đồng thời hệ thống hiển thị số lượng đối tượng và số vi phạm đang được ghi nhận tại thời điểm hiện tại. Người dùng có thể điều chỉnh ngưỡng độ tin cậy để lọc các phát hiện có xác suất thấp, từ đó quan sát ảnh hưởng của ngưỡng đến kết quả hiển thị.

Chế độ phát hiện AI đặc biệt hữu ích khi cần kiểm tra trực quan cách mô hình phản ứng với từng khung hình, trong khi chế độ video gốc phù hợp hơn cho thao tác tua, tạm dừng và đối chiếu bằng trình phát chuẩn của trình duyệt. Hai chế độ này bổ sung cho nhau: chế độ AI thể hiện quá trình suy luận, còn chế độ video gốc hỗ trợ kiểm chứng và phê duyệt kết quả.



Hình 17: Chế độ phát hiện AI hiển thị bounding box và thông tin suy luận trên video.

5.3 Triển khai

Hệ thống được triển khai trên Google Cloud theo mô hình container hóa. Mỗi thành phần chính của ứng dụng, bao gồm frontend Next.js và các dịch vụ backend, được đóng gói thành Docker image. Khi lập trình viên đẩy mã nguồn lên GitHub, GitHub Actions được kích hoạt để xây dựng image, gắn nhãn theo commit SHA và đưa image lên Artifact Registry. Sau đó các workload trên Google Kubernetes Engine Autopilot được cập nhật để sử dụng phiên bản image mới.

Môi trường chạy trên Google Cloud sử dụng GKE Autopilot tại khu vực `asia-southeast1`. Traefik là điểm vào công khai duy nhất của hệ thống. Chứng chỉ TLS được cấp thông qua Let's Encrypt và cert-manager, giúp người dùng truy cập ứng dụng bằng giao thức HTTPS. Các dịch vụ nội bộ như Orchestration, Notification và Inference Worker không được mở trực tiếp ra Internet mà chỉ được gọi thông qua luồng định tuyến và giao tiếp nội bộ trong hệ thống.

Ứng dụng đã được gắn tên miền DuckDNS và có thể truy cập tại [Helmet Violation Detection](https://dtat-nthv.duckdns.org).

Trong kiến trúc hiện tại, Supabase được sử dụng như dịch vụ bên ngoài cho cơ sở dữ liệu, xác thực, lưu trữ tệp và realtime. Redis được triển khai trong cụm Kubernetes để phục vụ hàng đợi tác vụ

và bộ nhớ đệm. Cách triển khai này phù hợp với mục tiêu demo và staging ban đầu: giảm số lượng dịch vụ cloud phải quản lý, chỉ sử dụng một điểm vào công khai, và vẫn đảm bảo các thành phần chính của hệ thống có thể được triển khai, cập nhật và rollback thông qua pipeline CI/CD.

6 Kết luận

6.1 Tóm tắt Kết quả

Tóm tắt các kết quả đạt được chính của đề án so với các mục tiêu ban đầu đề ra ở Chương 1.

6.2 Hạn chế

Thăng thẩn nhìn nhận và chỉ ra các hạn chế kỹ thuật hiện tại của mô hình phát hiện vi phạm và hệ thống ứng dụng triển khai thực tiễn (ví dụ: tốc độ suy luận, khả năng nhận diện ban đêm, độ trễ khi stream video, v.v.).

6.3 Hướng phát triển

Đề xuất các ý tưởng, giải pháp cải tiến hiệu năng mô hình, nâng cấp các tính năng cho ứng dụng hoặc hướng phát triển mở rộng trong tương lai.

Tài liệu

- [1] Lê Hoài Bắc and Tô Hoài Việt. *Cơ sở trí tuệ nhân tạo*. Nhà xuất bản khoa học và kỹ thuật, 2014.
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [3] Emma Pierson and Peyrin Kao. Cs 188: Introduction to artificial intelligence (Fall 2025), 2025. Accessed: November 3, 2025.
- [4] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [5] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, 2015.
- [6] Ultralytics. Ultralytics yolo documentation, 2024.
- [7] Wenhai Wang, Yifeng Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Rt-detr: Real-time detection transformer. *arXiv preprint arXiv:2304.08069*, 2023.